

Formulation du problème — Planification des laboratoires (Loyola)

Étape 6.1 du processus d’optimisation — document de référence versionné.
Il fige le périmètre mathématique du problème : ensembles, paramètres, variables de décision, contraintes dures, contraintes souples (pénalités) et fonction objectif. Toute évolution du modèle **doit** être répercutée ici en incrémentant la version ci-dessous.

Champ	Valeur
Version du document	1.0.0
Date	2026-06-18
Solveur	Google OR-Tools CP-SAT (<code>ortools</code> 9.15)
Fichier modèle	<code>pipeline.py</code> (fonction <code>solve</code>)
Couche QA	<code>data_quality.py</code> (Étape 6.2)
Mesure KPI	<code>kpi_report.py</code> (Étape 6.6)

0. Vue d’ensemble — architecture en 2 étages

La résolution est **séquentielle**, ce qui est essentiel pour interpréter le mot « OPTIMAL » renvoyé par le solveur :

1. **Étage heuristique glouton** (`form_groups` , ~l.1232) — décide, matière par matière : la **composition des groupes**, le **jour**, le **bloc horaire** et la **salle**. Ces choix sont figés avant d’appeler le solveur.
2. **Étage exact CP-SAT** (`solve` , ~l.3284) — pour ce squelette figé, décide la **semaine** de chaque séance afin d’étaler les séances de façon optimale.

 **Conséquence** : « OPTIMAL » signifie étalement hebdomadaire optimal sur un squelette jour/bloc/salle fixé, **pas** un optimum global du problème de planification. C’est un choix d’ingénierie assumé (rapidité, lisibilité) ; il est documenté ici pour éviter toute sur-interprétation.

1. Ensembles (Sets)

Symbole	Description	Source code
A	Étudiants (AlumnoID) inscrits à au moins une matière de labo	<code>subject_students</code>
M	Matières de laboratoire	clés de <code>subject_students</code>
G	Groupes formés (un groupe $\subset$ une matière)	<code>all_groups</code>
S	Séances à planifier (chaque groupe a <code>num_sessions</code> séances)	<code>sessions</code> dans <code>solve</code>
D	Jours ouvrés =	<code>DAYS</code> , <code>DAY_IDS</code>
B	Blocs horaires (créneaux de la journée)	<code>ALL_BLOCKS</code> , <code>BLOCK_LABELS</code>
R	Salles de laboratoire	<code>LAB_CONFIG[...]</code> <code>['lab_rooms']</code>
<code>W_sem</code>	Semaines disponibles du semestre	voir §2

Convention métier : 1 crédit de pratique (P) $\Rightarrow$ 5 séances de laboratoire.

2. Paramètres (Parameters)

Paramètre	Valeur par défaut	Description
MIN_GROUP_SIZE	7	Taille minimale d'un groupe
PREFERRED_GROUP_SIZE	12	Taille cible d'un groupe
MAX_GROUP_SIZE	15	Taille maximale d'un groupe
RECOVERY_MIN_GROUP_SIZE	7	Taille min. lors du repli (recovery)
SEMESTER_1_WEEKS	14	Nombre de semaines du semestre 1
SEMESTER_2_WEEKS	20	Nombre de semaines du semestre 2
[min_week, max_week]	par groupe	Fenêtre de semaines admissibles d'une séance
FRIDAY_SOFT_CAP	125	Plafond souple de séances le vendredi
FRIDAY_BASE_PENALTY	8	Pénalité de base (heuristique) pour le vendredi
FRIDAY_OVERCAP_WEIGHT	10	Escalade par séance au-delà du plafond
HOLIDAYS	par semestre	Semaines/jours fériés exclus
SOLVER_TIME_LIMIT	300 s	Temps max par semestre
RANDOM_SEED	42	Graine fixe (reproductibilité, Étape 6.5)
SOLVER_RELATIVE_GAP	0.02	Arrêt à 2 % de l'optimum prouvé (Étape 6.5)

3. Variables de décision (Decision Variables)

L'**unique** famille de variables du modèle exact est la **semaine** de chaque séance :

`week_vars[s]` $\in$ `Domain(valid_weeks(s))` pour chaque séance `s` $\in$ `S`

- Type : `IntVar` construite par `model.NewIntVarFromDomain(...)`.
- `valid_weeks(s)` = semaines de `[min_week(s), max_week(s)]` privées des semaines fériées applicables au jour du groupe (`HOLIDAYS`).

Le jour (`day_idx`), le bloc (`block_id`), la salle (`lab_rooms`) et la composition (`student_ids`) sont des **données d'entrée** du modèle exact (fixés par l'étage glouton), **pas** des variables.

4. Contraintes dures (Hard Constraints)

Code	Description	Implémentation
C1	Deux groupes d'une même matière partageant le même créneau (jour, bloc) doivent avoir des semaines différentes .	<code>week_vars[i] != week_vars[j] $\forall$ paire de même (subject, day_idx, block_id)</code>
C4	Deux séances partageant la même salle au même créneau (jour, bloc) doivent avoir des semaines différentes (pas de double réservation de salle).	<code>week_vars[i] != week_vars[j] $\forall$ paire de même (room, day_idx, block_id)</code>
C5	Les séances d'un même groupe sont strictement ordonnées dans le temps.	<code>week_vars[s_{k+1}] > week_vars[s_k]</code> (séances triées par <code>session</code>)

Domaine : chaque `week_vars[s]` est restreinte aux semaines réellement disponibles (hors fériés) — c'est une contrainte dure implicite portée par le domaine de la variable.

5. Contraintes souples / Pénalités (Soft Constraints)

Modélisées comme termes pondérés minimisés dans l'objectif.

Terme	Poids	But
<code>first_excess</code>	100	Pénalise les séances démar- rant après <code>min_week</code> (com- mencer tôt).
<code>last_deficit</code>	100	Pénalise les séances finissant avant <code>max_week</code> (utiliser toute la fenêtre).
<code>gap_deviations</code>	200	Pénalise l'écart à l'intervalle idéal entre séances consécut- ives d'un groupe (étalement régulier).
<code>parity</code> (alternance)	50 (<code>PARITY_PENALTY_WEIGHT</code>)	Favorise l'alternance pair/im- pair des semaines entre groupes (équilibre hebdo).
<code>c4_reserved</code>	100 000	Pénalité (quasi-dure) si une séance tombe sur une se- maine où la salle est réservée par une activité externe .

Le **goulot du vendredi** est traité en amont, dans l'étage glouton
(`friday_placement_penalty`), **pas** dans le solveur : c'est une pénalité de
score de placement qui détourne les groupes vers Lun→Jeu quand une
alternative faisable existe, sans jamais interdire le vendredi.

6. Fonction objectif (Objective)

```

minimize  Σ ( terme_i × poids_i )
          = 100 · Σ first_excess
          + 100 · Σ last_deficit
          + 200 · Σ gap_deviations
          + 50 · Σ parity
          + 100000 · Σ c4_reserved

```

Implémentation : `model.Minimize(total)` où
`total = Σ var · poids ( objective_terms )`.

Interprétation des poids : l'étalement régulier (gap, 200) prime sur le respect
des bornes de fenêtre (100) ; l'évitement des salles réservées externes
(100 000) est pratiquement inviolable ; l'alternance (50) est un réglage fin.

7. Diagnostic d'infaisabilité (Étape 6.4)

Si le solveur renvoie `INFEASIBLE`, `diagnose_infeasibility(...)` est appelée pour **identifier la ou les contraintes qui cassent** (domaines de semaines vides, créneaux sur-souscrits salle/matière), et un solveur de **repli** (recovery) relâche les regroupements pour récupérer une solution faisable (`_recovered = True`).

8. Réglage & reproductibilité du solveur (Étape 6.5)

Centralisés dans `configure_solver(...)` :

`random_seed = 42`, `relative_gap_limit = 0.02`,
`max_time_in_seconds = SOLVER_TIME_LIMIT`, `log_search_progress` paramétrable,
 plus un **démarrage à chaud** (`AddHint` via `add_week_hints`) qui propose des semaines régulièrement espacées.

9. Mesure de la qualité (Étape 6.6)

`kpi_report.py` calcule à chaque exécution : **% de placement**, nombre de groupes / overflow, **équilibre par jour** (suivi vendredi), **occupation des salles**, et **statistiques solveur** (statut, objectif, gap, temps). Sortie :
`reports/kpi_report.{json,txt}`.

Journal des versions

Version	Date	Changement
1.0.0	2026-06-18	Formalisation initiale (ensembles, paramètres, variables, C1/C4/C5, pénalités, objectif).